

区块链 Lab5 实验报告

学号: 2213600 姓名: 刘阳 专业: 信安法
学号: 2210977 姓名: 陈恩宝 专业: 信安

一. 代码设计

(一) 智能合约

本次实验设置智能合约要求 Solidity 编译器的版本至少为 0.4.0

```
pragma solidity >=0.4.0;
```

定义结构体 Debt 存储债务主体之间的债务 amount, amount 设置为 uint32 型, 并使用嵌套映射结构来存储债务关系, 可以直接使用 Debts[debtor][creditor].amount 来获取债务人 debtor 欠债权人 creditor 的金额

```
struct Debt
{
    uint32 amount;
}

mapping(address =>mapping(address => Debt)) internal Debts;
```

lookup 函数的实现就依靠上述的 Debts[debtor][creditor].amount

```
function lookup(address debtor, address creditor) public view returns
(uint32 ret)
{
    ret = Debts[debtor][creditor].amount;
}
```

add_IOU 函数的实现过程中, 需要检验债权人和债务人不能为同一人、金额必须为正数以及路径应当正确, 这些通过 require 实现

```
require(creditor != debtor, "Debtor and Creditor cannot be the same
person!");
require(amount > 0, "amount must be greater than 0 !");
require(creditor == path[0] && debtor == path[path.length - 1],
"The path is wrong!");
```

在设置参数时添加 minamount 表示循环中的最小代价, 因此当其为 0 时, 直接对债权人和债务人之间的债务增加 amount, 其他债务关系不用改动

```
if (minamount == 0) {
    debt.amount += amount;
    return;
```

```
}
```

当 minamount 不是 0 时，循环上的所有债务关系都要减去 minamount，只有债权人债务人之间的债务增加 amount，这里使用 for 循环实现

```
uint32 minAmount = minamount; // 初始最小金额为 minamount

// 计算路径上的债务金额，减少每个路径上金额
for (uint256 i = 0; i < path.length - 1; i++) {
    address currentCreditor = path[i];
    address nextCreditor = path[i + 1];

    // 判断路径上的债务是否存在并且是否足够扣除最小金额
    Debt storage intermediateDebt =
    Debts[currentCreditor][nextCreditor];
    require(intermediateDebt.amount >= minamount, "The debt
    does not enough to deduct the minamount!");

    // 更新路径债务金额
    intermediateDebt.amount -= minAmount;
}

// 增加债务金额，扣除已处理的最小金额
debt.amount += amount - minAmount;
```

(二) 客户端

在客户端除了实现指定的四个函数，还额外实现了两个函数来辅助实现特定的四个函数

1. getAllData()从区块链智能合约中获取符合条件的数据

这个函数调用 getAllFunctionCalls 获取所有执行 add_IOU 方法的交易记录，并使用 dataExtractor 从每个调用中提取相关数据，最后使用 new Set 实现去重，将结果转为数组返回

```
function getAllData(dataExtractor, condition) {
    const Calls = getAllFunctionCalls(contractAddress, 'add_
    IOU', condition);
    const Result = Calls.map(Call => dataExtractor(Call)).flat();
    return Array.from(new Set(Result));
}
```

2. getCreditors()获取特定用户的所有债权人

首先使用 `getAllData` 提取调用中的第一个参数 `creditor`，然后过滤得到当前用户实际欠款的债权人（及判断用户对债权人的欠款金额 >0 ）

```
function getCreditors(user) {
    const Creditors = getAllData(Call => [Call.args?.[0].toLowerCase()], null);
    return Creditors.filter(creditor => BlockchainSplitwise.lookup(user, creditor).toNumber() > 0);
}
```

3. `getUsers()`获取区块链中曾经作为债权人或债务人的所有用户

通过调用 `getAllData` 提取 `add_IOU` 调用 `Call.from` 即债务人、`Call.args?.[0]`即债权人，去重之后返回结果

```
function getUsers() {
    return getAllData(Call => [Call.from.toLowerCase(), Call.args?.[0].toLowerCase()], null);
}
```

4. `getTotalOwed()`计算用户当前欠款总金额

首先调用 `getAllData` 提取用户的所有债权人，然后遍历债权人，累加所有欠款金额，返回总和

```
function getTotalOwed(user) {
    const Creditors = getAllData(Call => [Call.args?.[0].toLowerCase()], null);
    let totalOwed = 0;
    Creditors.forEach(creditor => {
        totalOwed += BlockchainSplitwise.lookup(user, creditor).toNumber();
    });
    return totalOwed;
}
```

5. `getLastActive()`获取用户最后一次与合约交互的时间戳

首先使用 `getAllData` 检索用户相关的 `add_IOU` 调用，根据 `from` 或 `args?.[0]` 是否等于用户地址过滤出时间戳，然后使用 `Math.max()` 提取时间戳中的最大值，返回最新的交互时间。

```
function getLastActive(user) {
    const timeStamp = getAllData(Call => [Call.from.toLowerCase() === user.toLowerCase() ||
```

```

Call. args?. [0]. toLowerCase()      ===      user. toLowerCase()      ?
[Call.timestamp] : [],      Call => Call.from. toLowerCase() ===
user. toLowerCase()      ||      Call. args?. [0]. toLowerCase()      ===
user. toLowerCase());
      return timeStamp.length ? Math.max(...timeStamp) : null;
}

```

6. add_IOU()向系统中添加一个新的欠条

首先获取当前账户地址作为债务人，然后确保债权人和债务人不是同一人、金额为正整数

```

const debtor = web3.eth.defaultAccount;
if (debtor === creditor) {
    alert("Debtor and Creditor cannot be the same person!");
    return;
}
const parsedAmount = parseInt(amount, 10);
if (isNaN(parsedAmount) || parsedAmount <= 0) {
    alert("Amount must be a positive number!");
    return;
}

```

再调用 doBFS 查找从债权人到债务人的路径，如果找到路径：计算路径中的最小债务金额 minDebt，使用该最小债务金额与当前传入的金额 parsedAmount 中较小的一个，作为最终的债务金额。如果未找到路径，直接传入空路径和最小金额为 0。最后调用区块链合约 BlockchainSplitwise.add_IOU 添加欠条。

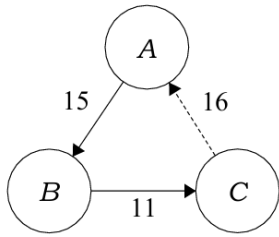
```

const path = doBFS(creditor, debtor, getCreditors);
if (path) {
    let minDebt = Infinity;
    for (let i = 1; i < path.length; i++) {
        const debt = BlockchainSplitwise.lookup(path[i - 1],
path[i]).toNumber();
        minDebt = Math.min(minDebt, debt);
    }
    const finalDebt = Math.min(minDebt, parsedAmount);
    return BlockchainSplitwise.add_IOU(creditor,
parsedAmount, path, finalDebt);
}
return BlockchainSplitwise.add_IOU(creditor, parsedAmount, [],
0);

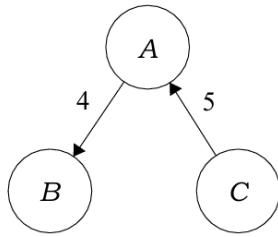
```

二. 实验效果

按照指导书上的原理图进行操作



becomes. . .



开始之前的账户信息：

Blockchain Splitwise

Add IOU

Address of person you owe:

Amount you owe them:

Add IOU

Users

My Account

Ox78cdadb285930ee34b44db917e369360d3c6b312

Total Owed: \$0

Last Activity: unknown

Wallet Addresses

1 Ox78cdadb285930ee34b44db917e369360d3c6b312

2 Ox78cdadb285930ee34b44db917e369360d3c6b312

3 Ox78cdadb285930ee34b44db917e369360d3c6b312

4 Ox78cdadb285930ee34b44db917e369360d3c6b312

5 Ox78cdadb285930ee34b44db917e369360d3c6b312

6 Ox78cdadb285930ee34b44db917e369360d3c6b312

7 Ox78cdadb285930ee34b44db917e369360d3c6b312

8 Ox78cdadb285930ee34b44db917e369360d3c6b312

9 Ox78cdadb285930ee34b44db917e369360d3c6b312

10 Ox78cdadb285930ee34b44db917e369360d3c6b312

A 欠 B，欠款为 15\$

Blockchain Splitwise

Add IOU

Address of person you owe:

Amount you owe them:

Add IOU

Users

My Account

Ox78cdadb285930ee34b44db917e369360d3c6b312

Total Owed: \$15

Last Activity: 12/2/2024, 8:50:13 PM

Wallet Addresses

1 Ox78cdadb285930ee34b44db917e369360d3c6b312

2 Ox78cdadb285930ee34b44db917e369360d3c6b312

3 Ox78cdadb285930ee34b44db917e369360d3c6b312

4 Ox78cdadb285930ee34b44db917e369360d3c6b312

5 Ox78cdadb285930ee34b44db917e369360d3c6b312

6 Ox78cdadb285930ee34b44db917e369360d3c6b312

7 Ox78cdadb285930ee34b44db917e369360d3c6b312

8 Ox78cdadb285930ee34b44db917e369360d3c6b312

9 Ox78cdadb285930ee34b44db917e369360d3c6b312

10 Ox78cdadb285930ee34b44db917e369360d3c6b312

B 欠 C，欠款为 11\$

Blockchain Splitwise

Add IOU

Address of person you owe:

Amount you owe them:

Add IOU

Users

- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312

My Account

0x63962d5ee733a37548cef006f695c7dab09bcf8

Total Owed: \$11

Last Activity: 12/2/2024, 8:50:48 PM

Wallet Addresses

- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8

C 欠 A，欠款为 16\$

Blockchain Splitwise

Add IOU

Address of person you owe:

0x78cdadb285930ee34b44db917e369360d3c6b312

Amount you owe them:

16

Add IOU

Users

- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312

My Account

0x4ef0682eec7f7327e7997278555afc231ae58328

Total Owed: \$0

Last Activity: 12/2/2024, 8:50:48 PM

Wallet Addresses

- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8
- 0x4ef0682eec7f7327e7997278555afc231ae58328
- 0x78cdadb285930ee34b44db917e369360d3c6b312
- 0x63962d5ee733a37548cef006f695c7dab09bcf8

生成循环，最小金额为 11\$，因此路径上的金额都要减去 11\$，得到结果如下：

My Account

0x78cdadb285930ee34b44db917e369360d3c6b312

Total Owed: \$4

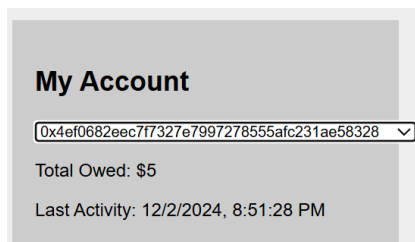
Last Activity: 12/2/2024, 8:51:28 PM

My Account

0x63962d5ee733a37548cef006f695c7dab09bcf8

Total Owed: \$0

Last Activity: 12/2/2024, 8:50:48 PM



三 . 实验心得

为了减少 gas 损耗，我进行了以下操作：

1. 对于增加欠条的信息，首先在客户端进行最小债务金额的计算和路径筛选，之后再传递给合约，以减少在合约中的计算
2. 合约中的存储操作非常昂贵，因此减少频繁的写操作，使用 memory 引入对于数组的操作，以减少存储成本